

1)Producer Consumer

```
#include<stdio.h>
int empty,full=0,size,in=0,out=0;
int buffer[100],item;
void display(){
    if(full==0){
        printf("The queue is empty:");
        return;
    }
    printf("The queue is: ");
    int i=out;
    for(int count=0;count<full;count++){
        printf("%d", buffer[i]);
        i=(i+1)%size;
    }
    printf("\n");
}
void producer(){
    if(full==size){
        printf("The queue is full:\n");
        return;
    }
    printf("Enter the item: ");
    scanf("%d",&item);
    buffer[in]=item;
    in=(in+1)%size;
    full++;
    empty--;
    printf("Item %d inserted: ",item);
    display();
}
void consumer(){
    if(full==0){
        printf("The queue is empty:");
        return;
    }
    item=buffer[out];
    out=(out+1)%size;
    full--;
    empty++;
    printf("Item %d consumed ",item);
    display();
}
int main(){
```

```

int choice;
printf("Enter the size of the queue: ");
scanf("%d",&size);
empty=size;
while(1){

    printf("\n1.Producer\n2.Consumer\nExit\n");
    printf("\nEnter the choice: ");

    scanf("%d",&choice);

    if(choice==1){
        producer();
        display();
    }
    else if(choice==2){
        consumer();
        display();
    }
    else if(choice ==3){
        break;
    }
    else{
        printf("Invalid choice");
    }
}
return 0;
}
2)Bankers Algo
#include<stdio.h>
int main(){
    int np,nr;
    printf("Enter the no of processes:");
    scanf("%d",&np);

    printf("Enter the number of resources: ");
    scanf("%d",&nr);

    int alloc[10][10],max[10][10],need[10][10],avail[10];

    printf("Enter the allocated matrix:\n");
    for(int i=0;i<np;i++){
        for(int j=0;j<nr;j++){

```

```

        scanf("%d",&alloc[i][j]);
    }
}

printf("Enter the maximum matrix: \n");
for(int i=0;i<np;i++){
    for(int j=0;j<nr;j++){
        scanf("%d",&max[i][j]);
    }
}
printf("Enter the available Matrix: \n");
for(int i=0;i<nr;i++){
    scanf("%d",&avail[i]);
}

printf("Need Matrix: \n");
for(int i=0;i<np;i++){
    for(int j=0;j<nr;j++){
        need[i][j]=max[i][j]-alloc[i][j];
        printf("%d ",need[i][j]);
    }
    printf("\n");
}
//safety algo
int finish[10]={0};
int safe[10];
int count=0;

while(count<np){

    int found = 0;
    for(int p=0;p<np;p++){
        if(finish[p]==0){
            int canRun=1;
            for(int r=0;r<nr;r++){
                if(need[p][r]>avail[r]){
                    canRun=0;
                    break;
                }
            }
            if(canRun){
                for(int r=0;r<nr;r++){
                    avail[r]+=alloc[p][r];
                }
            }
        }
    }
    count++;
}

```

```

        safe[count]=p;
        count++;
        finish[p]=1;
        found=1;
    }
}
}
if(found==0){
    printf("Unsafe state:");
    return 0;
}
}
printf("Safe Sequence: \n");
for(int i=0;i<np;i++){
    printf("P%d ",safe[i]);
}
printf("\n");
return 0;

```

}
 3)Process Scheduling

3a)FCFS

//fsfc

#include<stdio.h>

int main(){

int n ;

printf("Enter the number of processes: ");

scanf("%d",&n);

int at[n],bt[n],ct[n],tat[n],wt[n],p[n];

for(int i=0;i<n;i++){

p[i]=i+1;

printf("Process P%d",p[i]);

printf("Enter the Arrival Time:");

scanf("%d",&at[i]);

printf("Enter the burst time:");

scanf("%d",&bt[i]);

}

//bubble sort

for(int i=0;i<n-1;i++){

for(int j=0;j<n-i-1;j++){

if(at[j]>at[j+1]){

int temp=at[j];

```

        at[j]=at[j+1];
        at[j+1]=temp;

        temp=bt[j];
        bt[j]=bt[j+1];
        bt[j+1]=temp;

        temp=p[j];
        p[j]=p[j+1];
        p[j+1]=temp;
    }
}
int time=0;
float totalWt=0,totalTat=0;
for(int i=0;i<n;i++){
    if(time<-at[i])time=at[i];

    time+=bt[i];
    ct[i]=time;
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];

    totalWt+=wt[i];
    totalTat+=tat[i];

}
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++){
    printf("%d\t%d\t%d\t%d\t%d\t%d\t\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
}
printf("Average total wt %.2f",totalWt/n);
printf("Average total tat %.2f",totalTat/n);
return 0;
}
3b)SJF
#include<stdio.h>
int main(){
    int n;
    printf("Enter the number of process:");
    scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n],done[n];
    for(int i=0;i<n;i++){
        done[i]=0;

```

```

printf("Process %d ",i+1);
printf("Enter the Arrival Time:");
scanf("%d",&at[i]);
printf("Enter the burst time:");
scanf("%d",&bt[i]);
}
int time=0,completed=0;
while(completed<n){
    int min=99999,idx=-1;
    for(int i=0;i<n;i++){
        if(at[i]<=time && !done[i] && bt[i]<min){
            min=bt[i];
            idx=i;
        }
    }
    if(idx== -1){
        time++;
    }else{
        wt[idx]=time-at[idx];
        time+=bt[idx];
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        done[idx]=1;
        completed++;
    }
}

float totalWt=0,totalTat=0;
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++){
    printf("%d\t%d\t%d\t%d\t%d\t%d\t\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
    totalWt+=wt[i];
    totalTat+=tat[i];
}
printf("Average total wt %.2f",totalWt/n);
printf("Average total tat %.2f",totalTat/n);
return 0;

}
3c)Round Robin
#include<stdio.h>
int main(){
    int n;

```

```

printf("enter the no of processes:");
scanf("%d",&n);
int at[n],bt[n],ct[n],tat[n],wt[n],rt[n];
for(int i=0;i<n;i++){

printf("Process %d ",i+1);
printf("Enter the Arrival Time:");
scanf("%d",&at[i]);
printf("Enter the burst time:");
scanf("%d",&bt[i]);
rt[i]=bt[i];
}
int tq;
printf("Enter the time quantum:");
scanf("%d",&tq);

int time=0,completed=0;
while(completed<n){
int flag=0;
for(int i=0;i<n;i++){
if(rt[i]>0 && at[i]<time){
flag=1;

if(rt[i]>tq){
time+=tq;
rt[i]-=tq;
}else{
time+=rt[i];
ct[i]=time;
tat[i]=ct[i]-at[i];
wt[i]=tat[i]-bt[i];
rt[i]=0;
completed++;
}
}
}
if(!flag)time++;
}
float totalWt=0,totalTat=0;
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++){
printf("%d\t%d\t%d\t%d\t%d\t%d\t\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
totalWt+=wt[i];
totalTat+=tat[i];
}

```

```

    }
    printf("Average total wt %.2f",totalWt/n);
    printf("Average total tat %.2f",totalTat/n);
    return 0;
}
3d)Priority
#include<stdio.h>
int main(){
    int n;
    printf("Enter the no of process:");
    scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n],pr[n],done[n];
    for(int i=0;i<n;i++){
        done[i]=0;
        printf("ProcessP%d ",i+1);
        printf("arrival time ");
        scanf("%d",&at[i]);
        printf("burst time ");
        scanf("%d",&bt[i]);
        printf("priority ");
        scanf("%d",&pr[i]);
    }
    int time=0,completed=0;
    while(completed<n){
        int min=99999, idx=-1;
        for(int i=0;i<n;i++){
            if(at[i]<=time && !done[i] && pr[i]<min){
                min=pr[i];
                idx=i;
            }
        }
        if(idx== -1){
            time++;
        }else{
            wt[idx]=time-at[idx];
            time+=bt[idx];
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            done[idx]=1;
            completed++;
        }
    }
    float totalWt=0,totalTat=0;

```



```

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i=0;i<n;i++){
        printf("%d\t%d\t%d\t%d\t%d\t%d\t\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
        totalWt+=wt[i];
        totalTat+=tat[i];
    }
    printf("Average total wt %.2f",totalWt/n);
    printf("Average total tat %.2f",totalTat/n);
    return 0;
}

```

4) Page Replacement

4a) FIFO

```

//fifo page
#include<stdio.h>
int main(){
    int n,frames;
    printf("enter the reference length:");
    scanf("%d",&n);
    printf("Enter the no of frames:");
    scanf("%d",&frames);
    int pages[100],f[10];
    printf("Enter the pages:");
    for(int i=0;i<n;i++){
        scanf("%d",&pages[i]);
    }
    //initialize all frames as empty
    for(int i=0;i<frames;i++){
        f[i]=-1;
    }
    printf("\nPages\t");
    for(int i=0;i<frames;i++)printf("F%d",i+1);
    printf("status\n");
    int pos=0,fault=0;
    for(int i=0;i<n;i++){
        int found =0;
        for(int j=0;j<frames;j++){
            if(f[j]==pages[i]){
                found=1;
                break;
            }
        }
        printf("%d\t",pages[i]);
        if(found){
            for(int j=0;j<frames;j++){

```

```

        if(f[j]==-1){
            printf("-\t");
        }else{
            printf("%d\t",f[j]);
        }
    }
    printf("H\n");
}
else{

    f[pos]=pages[i];
    pos=(pos+1)%frames;
    fault++;

    for(int j=0;j<frames;j++){
        if(f[j]==-1){
            printf("-\t");
        }else{
            printf("%d\t",f[j]);
        }
    }
    printf("F\n");
}
}
printf("total page fault %d",fault);
return 0;

```

}

4b)LRU

```
#include<stdio.h>
```

```
int main(){
```

```
    int n,frames;
```

```
    printf("Enter the reference length:");
```

```
    scanf("%d",&n);
```

```
    printf("Enter the no of frames:");
```

```
    scanf("%d",&frames);
```

```
    int pages[100],f[10],time[10];
```

```
    printf("Enter the pages:");
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&pages[i]);
```

```
    }
```

```
    //initialize all frames as empty
```

```
    for(int i=0;i<frames;i++){
```

```
        f[i]=-1;
```

```
        time[i]=0;
```

```

}
int count=0,pos=0,fault=0;
printf("\nPages\t");
for(int i=0;i<frames;i++)printf("F%d\t",i+1);
printf("status\n");
for(int i=0;i<n;i++){
    int found =0;
    for(int j=0;j<frames;j++){
        if(f[j]==pages[i]){
            count++;
            time[j]=count;
            found=1;
            break;
        }
    }
    printf("%d\t",pages[i]);
    int min=time[0];
    if(found){
        for(int j=0;j<frames;j++){
            if(f[j]==-1){
                printf("-\t");
            }else{
                printf("%d\t",f[j]);
            }
        }
        printf("H\n");
    }else{

        for(int j=1;j<frames;j++){
            if(time[j]<min){
                min=time[j];
                pos=j;
            }
        }
        f[pos]=pages[i];
        count++;
        time[pos]=count;
        fault++;
        for(int j=0;j<frames;j++){
            if(f[j]==-1){
                printf("-\t");
            }else{
                printf("%d\t",f[j]);
            }
        }
    }
}

```

```

    }
    printf("F\n");
}
}
printf("total page fault %d",fault);
return 0;
}
4c)Optimal
#include<stdio.h>
int main(){
    int n,frames;
    int fault=0;
    printf("Enter the reference length:");
    scanf("%d",&n);
    printf("Enter the no of frames:");
    scanf("%d",&frames);
    int pages[100],f[10];
    printf("Enter the pages:");
    for(int i=0;i<n;i++){
        scanf("%d",&pages[i]);
    }
    //initialize all frames as empty
    for(int i=0;i<frames;i++){
        f[i]=-1;
    }
    printf("\nPages\t");
    for(int i=0;i<frames;i++)printf("F%d\t",i+1);
    printf("Status\n");
    for(int i=0;i<n;i++){
        int found =0;
        for(int j=0;j<frames;j++){
            if(f[j]==pages[i]){
                found=1;
                break;
            }
        }
        printf("%d\t",pages[i]);
        int pos=-1,far=-1;

        if(found){
            for(int j=0;j<frames;j++){
                if(f[j]==-1){
                    printf("-\t");
                }else{

```

```

        printf("%d\t",f[j]);
    }
}
printf("H\n");
}else{
for(int j=0;j<frames;j++){
    if(f[j]==-1){
        pos=j;
        break;
    }
}
if(pos==-1){
    for(int j=0;j<frames;j++){
        int next=-1;
        for(int k=i+1;k<n;k++){
            if(f[k]==pages[k]){
                next=k;
                break;
            }
        }
        if(next==-1){
            pos=j;
            break;
        }
        if(next>far){
            far=next;
            pos=j;
        }
    }
}
}

f[pos]=pages[i];
fault++;
for(int j=0;j<frames;j++){
    if(f[j]==-1){
        printf("-\t");
    }else{
        printf("%d\t",f[j]);
    }
}
printf("F\n");
}
}

```

```

    printf("total page fault %d",fault);
    return 0;
}
5)Disk Scheduling
5a)sstf
#include<stdio.h>
#include<stdlib.h>
int main(){
    int n;
    printf("Enter the no of requests:");
    scanf("%d",&n);
    int req[100];
    printf("Enter the requests:");
    for(int i=0;i<n;i++){
        scanf("%d",&req[i]);
    }
    int head,total=0;
    printf("Enter the initial head position:");
    scanf("%d",&head);
    int pos=head;
    printf("Seek Sequence %d",pos);
    int visited[10]={0};

    for(int i=0;i<n;i++){
        int min=9999,index=-1;
        for(int j=0;j<n;j++){
            if(!visited[j]){
                int dist=abs(pos-req[j]);
                if(dist<min){
                    min=dist;
                    index=j;
                }
            }
        }
        visited[index]=1;
        total+=min;
        pos=req[index];
        printf("->%d",pos);
    }
    printf("Total head movement : %d", total);
    return 0;
}

```

```

5b)look
#include<stdio.h>
#include<stdlib.h>
int main(){
    int n;
    printf("Enter the no of requests:");
    scanf("%d",&n);
    int req[100];
    printf("Enter the requests:");
    for(int i=0;i<n;i++){
        scanf("%d",&req[i]);
    }
    int head;
    printf("Enter the initial head position:");
    scanf("%d",&head);
    //bubble sort
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(req[j]>req[j+1]){
                int temp=req[j];
                req[j]=req[j+1];
                req[j+1]=temp;
            }
        }
    }
    int split=0;
    int total =0;
    int pos=head;

    //find split
    for(int i=0;i<n;i++){
        if(req[i]>=head){
            split=i;
            break;
        }
    }
    printf("Seek Sequence %d ",pos);
    //right
    for(int i=split;i<n;i++){
        total+=abs(pos-req[i]);
        pos=req[i];
        printf("->%d",pos);
    }
    //reverse

```

```

    for (int i=split-1;i>=0;i--){
        total+=abs(pos-req[i]);
        pos=req[i];
        printf("->%d",pos);
    }
    printf("Total head movement: %d", total);
    return 0;
}

```

5c)cscan

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int main(){
```

```
    int n,disk_size;
```

```
    printf("Enter the disk_size:");
```

```
    scanf("%d",&disk_size);
```

```
    printf("Enter the no of requests:");
```

```
    scanf("%d",&n);
```

```
    int req[100];
```

```
    printf("Enter the requests:");
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&req[i]);
```

```
    }
```

```
    int head;
```

```
    printf("Emter the initial head position:");
```

```
    scanf("%d",&head);
```

```
//bubble sort
```

```
for(int i=0;i<n-1;i++){
```

```
    for(int j=0;j<n-i-1;j++){
```

```
        if(req[j]>req[j+1]){
```

```
            int temp=req[j];
```

```
            req[j]=req[j+1];
```

```
            req[j+1]=temp;
```

```
        }
```

```
    }
```

```
}
```

```
int pos=head,total=0;
```

```
int split=0;
```

```
for(int i=0;i<n;i++){
```

```
    if(req[i]>=head){
```

```
        split=i;
```

```
        break;
```

```
    }
```

```
}
```



```
printf("seek Sequence %d",pos);
```

```
//right
```

```
for(int i=split;i<n;i++){  
    total+=abs(pos-req[i]);  
    pos=req[i];  
    printf("->%d",pos);  
}
```

```
//jump to zero
```

```
total+= abs(pos-(disk_size-1));  
pos=disk_size-1;  
printf("->%d",pos);
```

```
total+= abs(pos-0);  
pos=0;  
printf("->%d",pos);
```

```
for(int i=0;i<split;i++){  
    total+=abs(pos-req[i]);  
    pos=req[i];  
    printf("->%d",pos);  
}  
printf("Total head movement :%d",total);  
return 0;
```

```
}
```

5d)using switch look and scan

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int main(){
```

```
    int choice;
```

```
    printf("1. SCAN\n");  
    printf("2. CSCAN\n");  
    printf("Enter your choice: ");  
    scanf("%d",&choice);
```

```
    int n,disk_size;
```

```
    printf("Enter disk size: ");  
    scanf("%d",&disk_size);
```

```
printf("Enter the no of requests: ");
scanf("%d",&n);
```

```
int req[100];
```

```
printf("Enter the requests: ");
```

```
for(int i=0;i<n;i++){
    scanf("%d",&req[i]);
}
```

```
int head;
```

```
printf("Enter initial head position: ");
scanf("%d",&head);
```

```
// Bubble Sort
```

```
for(int i=0;i<n-1;i++){
    for(int j=0;j<n-i-1;j++){
        if(req[j] > req[j+1]){
            int temp=req[j];
            req[j]=req[j+1];
            req[j+1]=temp;
        }
    }
}
```

```
int split=0;
```

```
for(int i=0;i<n;i++){
    if(req[i] >= head){
        split=i;
        break;
    }
}
```

```
int pos=head,total=0;
```

```
printf("Seek Sequence: %d",pos);
```

```
switch(choice){

    // SCAN

    case 1:

        // Right side

        for(int i=split;i<n;i++){

            total += abs(pos-req[i]);

            pos=req[i];

            printf(" -> %d",pos);
        }

        // Reverse direction

        for(int i=split-1;i>=0;i--){

            total += abs(pos-req[i]);

            pos=req[i];

            printf(" -> %d",pos);
        }

        break;

    // CSCAN

    case 2:

        // Right side

        for(int i=split;i<n;i++){

            total += abs(pos-req[i]);

            pos=req[i];

            printf(" -> %d",pos);
```

```

    }

    // Move to end

    total += abs(pos-(disk_size-1));

    pos=disk_size-1;

    printf(" -> %d",pos);

    // Jump to 0

    total += abs(pos-0);

    pos=0;

    printf(" -> %d",pos);

    // Continue from beginning

    for(int i=0;i<split;i++){

        total += abs(pos-req[i]);

        pos=req[i];

        printf(" -> %d",pos);
    }

    break;

default:

    printf("Invalid choice");
    return 0;
}

printf("\nTotal head movement: %d\n",total);

return 0;
}
5e)scan
#include<stdio.h>
#include<stdlib.h>

```

```

int main(){

    int n,disk_size;

    printf("Enter the disk size: ");
    scanf("%d",&disk_size);

    printf("Enter the number of requests: ");
    scanf("%d",&n);

    int req[100];

    printf("Enter the requests: ");

    for(int i=0;i<n;i++){
        scanf("%d",&req[i]);
    }

    int head;

    printf("Enter the initial head position: ");
    scanf("%d",&head);

    // Bubble Sort

    for(int i=0;i<n-1;i++){

        for(int j=0;j<n-i-1;j++){

            if(req[j] > req[j+1]){

                int temp=req[j];
                req[j]=req[j+1];
                req[j+1]=temp;
            }
        }
    }

    int split=0;

    // Find split position

    for(int i=0;i<n;i++){

```

```

        if(req[i] >= head){
            split=i;
            break;
        }
    }

    int pos=head,total=0;

    printf("\nSeek Sequence: %d",pos);

    // Move right side

    for(int i=split;i<n;i++){

        total += abs(pos-req[i]);

        pos=req[i];

        printf(" -> %d",pos);
    }

    // Move to disk end

    total += abs(pos-(disk_size-1));

    pos=disk_size-1;

    printf(" -> %d",pos);

    // Reverse direction

    for(int i=split-1;i>=0;i--){

        total += abs(pos-req[i]);

        pos=req[i];

        printf(" -> %d",pos);
    }

    printf("\nTotal head movement: %d\n",total);

    return 0;

```

```

}
6)fork 1
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
int main(){
    pid_t pid;
    pid=fork();
    if(pid<0){
        printf("fork failed");
        return 1;
    }else if(pid==0){
        printf("Child Process:\n");
        printf("Child Pid:%d\n",getpid());
        printf("parent pid:%d\n",getppid());
    }else{
        printf("parent process:\n");
        printf("child pid:%d\n",pid);
        printf("parent pid:%d\n",getpid());
    }
    sleep(300);
    return 0;
}

```

```

7)fork variable
#include<stdio.h>
#include<unistd.h>
int main(){
    int x;
    printf("Enter the value for x: ");
    scanf("%d",&x);
    pid_t pid;
    pid=fork();

    if(pid<0){
        printf("Fork failed!");
        return 1;
    }else if(pid==0){
        x=x+5;
        printf("Child process(x+5) : x=%d\n",x);
    }else{
        x=x-5;
    }
}

```

```

    printf("Parent process (x-5):x =%d\n",x);
}
return 0;
}

```

8)fork and wait

```

#include<stdio.h>
#include<sys/wait.h>
#include<unistd.h>
int main(){
    pid_t pid;
    pid=fork();
    if(pid<0){
        perror("fork failed");
        return 1;
    }
    else if(pid==0){
        printf("child:\n");
        printf("course code\n");
    }
    else{
        wait(NULL);
        printf("parent:\n");
        printf("os lab\n");
    }
    return 0;
}

```

9)fork multiple child processes with same parent

//to create multiple child process under same parent

```

#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main(){
    printf("Main Parent PID :%d\n\n",getpid());

    for(int i=0;i<3;i++){
        if(fork()==0){
            printf("CHild %d [PID :%d]->Parent [PID:%d]\n",i+1,getpid(),getppid());
            return 0;
        }
    }
    for(int i=0;i<3;i++){
        wait(NULL);
    }
}

```



```
    return 0;  
}
```